Week 13

Name:AKSHAY P
Mobile:

Data Structure Workouts
1. Learn what is Data Structure & Algorithms. 2. Learn the basics of Memory Allocation and Memory leak. 3. Learn the concept of Complexity Analysis. NB: The complexity of common operations of all data structures should be covered. 4. Learn about Asymptotic analysis (Big-O notation). 5. Learn the concepts of Array. Complete at least three sample workouts & do at least 3 problems from any competitive coding websites (Hacker Rank, Code Chef, Leet code, Algo Expert, etc.) 6. Learn the concepts of the Linked list. Complete at least three sample workouts a. Construction of Singly linked list & Doubly linked list. b. Convert array to a linked list c. Add a node at the end & beginning d. Delete node with the value specified e. Insert a node after & before a node with x data f. Print all elements by order & reverse by order g. Write a program to remove duplicates in a sorted singly linked list 7. Learn the concepts of String. Complete at least three sample workouts. Eg: Write a function to replace each alphabet in the given string with another alphabet occurring at the n-th position from each of them. 8. Learn about Linear Search & Binary Search. Complete at least 3 sample workouts in each of them 9. Learn the concepts of Recursion. Complete at least 3 sample workouts. 10. Learn about the applications of all structures you covered this week
An Array in JavaScript is a versatile and fundamental data structure that allows you to store and manipulate collections of elements. It is a linear, ordered list of values, and each element in the array is identified by its index, starting from zero. Arrays in JavaScript can hold various

data types, including numbers, strings, objects, and even other arrays. They offer a range of built-in methods for common operations such as adding, removing, and manipulating elements, making them a powerful tool for organizing and processing data in JavaScript applications.

Link to the folder containing code and screenshot of the output

---

A linked list in JavaScript is a data structure that consists of a sequence of elements, where each element points to the next one in the sequence through a reference or link. Unlike arrays, linked lists do not have a fixed size, and elements are stored in nodes that dynamically allocate memory.

In a basic singly linked list, each node contains two properties: the data value and a reference to the next node in the list. The last node typically points to null, indicating the end of the list. This dynamic structure allows for efficient insertion and deletion of elements, as nodes can be easily rearranged by updating references.

Linked lists are useful when the size of the data set is unknown or may change frequently. However, accessing elements in a linked list is generally slower than in an array, as traversal is required from the beginning to reach a specific element.

JavaScript does not have a built-in linked list implementation, but developers can create custom linked list structures using objects and references to achieve the desired functionality.

Link to the folder containing code and screenshot of the output

---

Certainly! A "string" in JavaScript refers to a sequence of characters enclosed within single (' ') or double (" ") quotes. Strings can include letters, numbers, symbols, and spaces. They are a fundamental data type in JavaScript and are used for representing and manipulating text. Various built-in methods and operators are available for working with strings, allowing developers to perform operations such as concatenation, slicing, searching, and more. Strings play a crucial role in web development, as they are commonly used for handling user input, displaying text on websites, and managing data in JavaScript programs.

Link to the folder containing code and screenshot of the output

---

Linear Search:

Linear search is a straightforward searching algorithm that traverses a list of elements sequentially until it finds the desired item or reaches the end of the list. It compares each element one by one with the target value, making it suitable for both sorted and unsorted arrays. However, its simplicity comes at the cost of efficiency, especially for large datasets.

Binary Search:

Binary search is an efficient searching algorithm, ideal for sorted arrays. It works by repeatedly dividing the search space in half, comparing the target value with the middle element, and eliminating half of the remaining elements in each iteration. This process continues until the target is found or the search space is empty. Binary search is notably faster than linear search for large datasets but requires the array to be sorted for optimal performance.

Link to the folder containing code and screenshot of the output

Recursion in JavaScript refers to the programming technique where a function calls itself during its execution. In a recursive function, the problem is solved by breaking it down into smaller, more manageable instances of the same problem. Each recursive call contributes to solving the overall problem, and the process continues until a base case is reached, preventing infinite recursion. Recursion is a powerful and elegant approach, particularly useful for solving problems that exhibit a recursive structure or when dealing with hierarchical data structures like trees.

Link to the folder containing code and screenshot of the output